Representation of Active Learning by Utilisation of the Typiclust Algorithm

Hissan Omar
K23136072
5CCSAMLF (CW2)

March 2026

1 Introduction

1.1 A summary of the problem

Modern machine learning models require large labelled datasets, but acquiring labels is often costly and time consuming. Active learning aims to reduce this problem by selecting the most informative samples for labelling.

Hacohen et al. (2022) proposes a representation-based approach that prioritises *representative* samples instead of uncertain ones. Traditional uncertainty-based methods often select redundant samples particularly in early stages.

The method in the paper introduces **typicality**, which measures how representative a sample is within a learned feature space. Using a model, feature embeddings are learned, clustered, and the most typical samples (closest to cluster centres) are selected. This approach is known as TPCRP and improves data efficiency under limited labelling budgets.

1.2 The significance of the problem in the broader context of the field

The significance of the problem is highlighted through how this work shifts active learning from uncertainty-driven to representation-driven selection. It is especially effective for high-dimensional datasets and provides an efficient solution when labelled data is limited.

2 Methodology

2.1 A concise description of the algorithm and its steps

The TPCRP algorithm selects informative samples through representation learning, clustering, and typicality-based selection:

1. Representation Learning: Train a model to obtain feature embeddings of the dataset. **2. Feature Extraction:** Map all images into a high dimensional embedding space. **3. Clustering:** Partition the embeddings into K clusters, where K is the labelling budget. **4. Typicality Computation:** For each sample, compute its typicality based on similarity to the neighbouring points. **5. Sample Selection:** Select the most typical sample from each cluster for labelling. **6. Model Training:** Train a classifier on the selected subset and finally evaluate performance.

2.2 Key concepts or techniques that are central to the method

Key concepts include **self-supervised representation learning, clustering in feature space, and density-based typicality**. All these methods enable selection of diverse and representative samples without requiring labels.

3 Results

3.1 Summarising the results presented in the paper

Method	Accuracy (%)
Random	31.9
Entropy	31.8
CoreSet	31.1
TPCRP (Typiclust)	**36.3**

Table 1: Results reported in the paper (CIFAR-10).

Method	Accuracy (%)
TPCRP	10.96
Centrality TPCRP	**12.43**
Random	12.24

Table 2: Results from my implementation.

The results according to the paper show that TPCRP outperforms random and uncertainty-based methods, achieving higher accuracy by selecting representative samples. This demonstrates the effectiveness of representation-based active learning under limited labelling budgets.

The results according to my implementation show that a similar trend is observed, with centrality-based TPCRP achieving the highest accuracy, followed by random sampling and standard TPCRP. Although overall accuracy is lower due to simplified training and limited training infrastructure, **the results confirm that incorporating centrality makes an improvement to TPCRP** and better captures cluster structure.

3.2 Key findings that demonstrate the algorithm's effectiveness.

TPCRP demonstrates strong effectiveness by consistently outperforming the baseline methods. The results show that selecting representative samples improves learning efficiency, while the centrality-based modification further enhances performance by capturing global structure.

3.3 The Modified Algorithm and it's Evaluation

The modified algorithm was made using the concept of **centrality-weighted-typicality.** It improves sample selection by incorporating the distance to cluster centroids alongside typicality. This encourages the selection of samples that are both locally representative and globally central leading to improved accuracy compared to the original TPCRP.

However this modification introduces additional computational cost and it's main problem is that it may bias selection towards dense regions, potentially overlooking informative boundary samples.

Overall, the tables above show that approach was successful, demonstrating improved performance. Future work could explore adaptive weighting between typicality and centrality or more efficient centrality estimation methods.

4 Plots

4.1 Visualisation of the results.

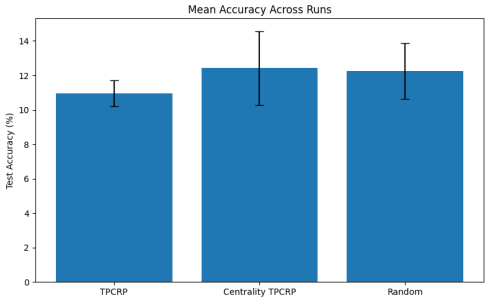

Figure 1: Mean test accuracy across runs for TPCRP, centrality-based TPCRP, and random sampling. Error bars represent standard deviation.

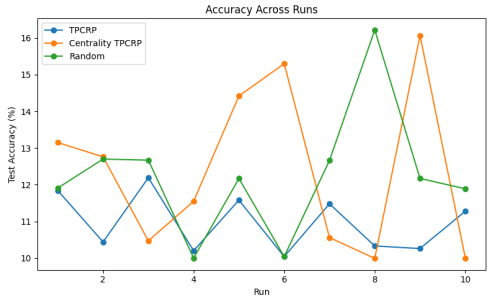

Figure 2: Test accuracy across repeated runs for TPCRP, centrality-based TPCRP, and random sampling.

5 Statistical Analysis

5.1 Statistical analysis that support the results

Across 10 runs, the centrality-based TPCRP achieved the highest mean accuracy **(12.43%)** compared to TPCRP **(10.96%)** and random sampling **(12.24%)**. Standard deviations indicate variability, with centrality TPCRP **(±2.15)** showing greater variability across runs than TPCRP **(±0.75)**. The plots further confirm this, where the bar chart highlights overall performance differences and the line plot shows fluctuations across runs, specifically in the centrality line, where volatility is much more prominent than the original TPCRP. This consistency in outperforming baseline methods suggests the modification is effective despite the increased variance.

5.2 The significance of the statistical findings in relation to the paper's claims

These results support the paper's claim that representation-based selection improves performance over baseline methods. The centrality-based modification further enhances this by improving average accuracy, although as stated in section 5.1, increased variance suggests a trade-off between performance and stability. This aligns with the paper's emphasis on selecting representative samples to maximise learning efficiency under limited labelling budgets.

6 Conclusion

6.1 Summarising the key takeaways from the paper

The paper demonstrates that representation-based active learning, specifically TPCRP, outperforms traditional uncertainty-based methods by selecting representative samples in feature space. By leveraging clustering and typicality, the approach ensures diverse and informative sample selection, leading to improved learning efficiency under limited labelling budgets. Overall, the method highlights the importance of representation quality in guiding effective sample selection.

6.2 The implications of the results and any potential future work

In conclusion, the results highlight the effectiveness of combining representation learning with sample selection strategies. The improved performance of the centrality-based modification suggests that incorporating global structure can enhance selection quality. Future work could explore alternative clustering methods such as DBSCAN, which does not require a predefined number of clusters and can better capture density variations. Additionally, adaptive weighting between typicality and centrality could further improve performance and stability.

7 Code Supplement

Here is the link to the repository. After cloning the repository, please take a look at the `README.md` file for instructions on installation and setup: `https://github.com/Hissan7/typiclust-active-learning`

5CCSAMLF CW2 Code Appendix

Original TPCRP and Modified Centrality-Based TPCRP Implementations

In [1]:
```
import os
import sys
import numpy as np
from pathlib import Path

def find_project_root(start: Path) -> Path:
    start = start.resolve()

    candidates = [start, *start.parents]
    for base in candidates:
        if (base / "src").is_dir():
            return base
        if (base / "typiclust-active-learning" / "src").is_dir():
            return base / "typiclust-active-learning"

    raise FileNotFoundError("Could not find project root containing 'src'.")

project_root = find_project_root(Path.cwd())
src_path = project_root / "src"

if str(src_path) not in sys.path:
    sys.path.insert(0, str(src_path))

print("Working directory:", Path.cwd())
print("Project root:", project_root)
print("Source path:", src_path)
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F

from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms
from torchvision.transforms import ToPILImage
import torchvision.models as models

from sklearn.cluster import KMeans
from sklearn.neighbors import NearestNeighbors
```

```
Working directory: /Users/omxr/Desktop/VSCodeProjects/5CCSAMLF
Project root: /Users/omxr/Desktop/VSCodeProjects/5CCSAMLF/typiclust-active-learning
Source path: /Users/omxr/Desktop/VSCodeProjects/5CCSAMLF/typiclust-active-learning/src
```

1. SimCLR Augmentations

In [2]:
```
class SimCLRTransform:
    def __init__(self, image_size: int = 32):
        color_jitter = transforms.ColorJitter(
            brightness=0.8,
            contrast=0.8,
            saturation=0.8,
            hue=0.2
        )

        self.base_transform = transforms.Compose([
            transforms.RandomResizedCrop(size=image_size, scale=(0.2, 1.0)),
            transforms.RandomHorizontalFlip(p=0.5),
            transforms.RandomApply([color_jitter], p=0.8),
            transforms.RandomGrayscale(p=0.2),
            transforms.ToTensor(),
        ])

    def __call__(self, x):
        x1 = self.base_transform(x)
        x2 = self.base_transform(x)
        return x1, x2
```

2. Contrastive loss

In [3]:
```
def nt_xent_loss(z1, z2, temperature: float = 0.5):
    batch_size = z1.size(0)

    z1 = F.normalize(z1, dim=1)
    z2 = F.normalize(z2, dim=1)

    z = torch.cat([z1, z2], dim=0)

    similarity = torch.matmul(z, z.T)
    similarity = similarity / temperature

    mask = torch.eye(2 * batch_size, device=z.device).bool()
    similarity = similarity.masked_fill(mask, float("-inf"))

    targets = torch.arange(batch_size, device=z.device)
    targets = torch.cat([targets + batch_size, targets], dim=0)

    loss = F.cross_entropy(similarity, targets)
    return loss
```

3. SimCLR model

In [4]:
```
class SimCLR(nn.Module):
    def __init__(self, projection_dim: int = 128):
        super().__init__()

        backbone = models.resnet18(weights=None)
```

4. SimCLR training

In [5]:
```python
from simclr.augmentations import SimCLRTransform
from simclr.simclr_model import SimCLR
from simclr.contrastive_loss import nt_xent_loss


class CIFAR10PairDataset:
    def __init__(self, root="./data", train=True, download=True, transform=None):
        self.dataset = datasets.CIFAR10(root=root, train=train, download=download)
        self.transform = transform
        self.to_pil = ToPILImage()

    def __len__(self):
        return len(self.dataset)

    def __getitem__(self, idx):
        image, _ = self.dataset[idx]

        if self.transform is None:
            raise ValueError("Transform must not be None for SimCLR training.")

        x1, x2 = self.transform(image)
        return x1, x2


def get_device():
    if torch.backends.mps.is_available():
        return "mps"
    if torch.cuda.is_available():
        return "cuda"
    return "cpu"


def train_simclr(
    epochs: int = 50,
    batch_size: int = 256,
    lr: float = 1e-3,
    temperature: float = 0.5,
    save_path: str = "models/simclr_resnet18.pth"
):
    device = get_device()
    print("SimCLR training device:", device)

    transform = SimCLRTransform(image_size=32)
    dataset = CIFAR10PairDataset(transform=transform)

    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=True,
        num_workers=0
    )

    model = SimCLR(projection_dim=128).to(device)

    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

    model.train()
    for epoch in range(epochs):
        running_loss = 0.0

        for x1, x2 in loader:
            x1 = x1.to(device)
            x2 = x2.to(device)

            _, z1 = model(x1)
            _, z2 = model(x2)

            loss = nt_xent_loss(z1, z2, temperature=temperature)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            running_loss += loss.item()

        avg_loss = running_loss / len(loader)
        print(f"Epoch {epoch + 1}/{epochs} | SimCLR Loss: {avg_loss:.4f}")

    os.makedirs(os.path.dirname(save_path), exist_ok=True)
    torch.save(model.state_dict(), save_path)
    print(f"Saved SimCLR model to {save_path}")

    return model
```

5. Embedding extraction

```python
from simclr.simclr_model import SimCLR


def get_device():
    if torch.backends.mps.is_available():
        return "mps"
    if torch.cuda.is_available():
        return "cuda"
    return "cpu"


def load_trained_simclr(model_path: str):
    device = get_device()
    model = SimCLR(projection_dim=128).to(device)
    model.load_state_dict(torch.load(model_path, map_location=device))
    model.eval()
    return model, device


def get_cifar10_plain_loader(train=True, batch_size=256):
    transform = transforms.Compose([
        transforms.ToTensor(),
    ])

    dataset = datasets.CIFAR10(
        root="./data",
        train=train,
        download=True,
        transform=transform
    )

    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=False,
        num_workers=0
    )

    return dataset, loader


def extract_embeddings(model_path: str, train=True):
    model, device = load_trained_simclr(model_path)
    dataset, loader = get_cifar10_plain_loader(train=train)

    all_embeddings = []

    with torch.no_grad():
        for images, _ in loader:
            images = images.to(device)
            h, _ = model(images)
            h = F.normalize(h, dim=1)
            all_embeddings.append(h.cpu())

    embeddings = torch.cat(all_embeddings, dim=0).numpy()
    return dataset, embeddings
```

6. Clustering

```python
def cluster_features(features, num_clusters: int):
    kmeans = KMeans(n_clusters=num_clusters, random_state=42, n_init=10)
    cluster_labels = kmeans.fit_predict(features)
    return cluster_labels
```

7. Typicality and Centrality

```python
def compute_typicality_of_points(cluster_features: np.ndarray, k: int = 20) -> np.ndarray:
    """
    Computes typicality for each point in a cluster
    typicality = 1 / average distance to k nearest neighbours
    """

    if len(cluster_features) == 1:
        return np.array([float("inf")])

    k = min(k, len(cluster_features) - 1)

    nbrs = NearestNeighbors(n_neighbors=k + 1)
    nbrs.fit(cluster_features)

    distances, _ = nbrs.kneighbors(cluster_features)

    neighbour_distances = distances[:, 1:]

    avg_distances = neighbour_distances.mean(axis=1)

    typicality = 1.0 / (avg_distances + 1e-8)

    return typicality


def compute_centrality(cluster_features: np.ndarray) -> np.ndarray:
    """
    Computes centrality = inverse distance to cluster centroid
    """

    centroid = np.mean(cluster_features, axis=0)

    distances = np.linalg.norm(cluster_features - centroid, axis=1)

    centrality = 1.0 / (distances + 1e-8)

    return centrality
```

8. Sample selection methods

```python
from typicality import compute_typicality_of_points,compute_centrality


def select_most_typical_per_cluster(
    features: np.ndarray,
    cluster_labels: np.ndarray,
    budget: int,
    min_cluster_size: int = 5
):
    selected_indices = []
    selected_set = set()

    unique_clusters = np.unique(cluster_labels)

    cluster_to_indices = {}
    for cluster_id in unique_clusters:
        indices = np.where(cluster_labels == cluster_id)[0]
        if len(indices) >= min_cluster_size:
            cluster_to_indices[cluster_id] = indices

    cluster_label_counts = {cluster_id: 0 for cluster_id in cluster_to_indices.keys()}

    while len(selected_indices) < budget and len(cluster_to_indices) > 0:
        min_count = min(cluster_label_counts.values())

        candidate_clusters = [
            cluster_id for cluster_id, count in cluster_label_counts.items()
            if count == min_count
        ]

        chosen_cluster = max(candidate_clusters, key=lambda cid: len(cluster_to_indices[cid]))
        cluster_indices = cluster_to_indices[chosen_cluster]

        available_indices = [idx for idx in cluster_indices if idx not in selected_set]

        if len(available_indices) < min_cluster_size:
            del cluster_to_indices[chosen_cluster]
            del cluster_label_counts[chosen_cluster]
            continue

        cluster_features = features[available_indices]
        k = min(20, len(available_indices) - 1)

        if k < 1:
            del cluster_to_indices[chosen_cluster]
            del cluster_label_counts[chosen_cluster]
            continue

        scores = compute_typicality_of_points(cluster_features, k=k)
        best_local_index = int(np.argmax(scores))
        best_global_index = available_indices[best_local_index]

        selected_indices.append(best_global_index)
        selected_set.add(best_global_index)
        cluster_label_counts[chosen_cluster] += 1

    return selected_indices


def select_weighted_typical_samples(
    features: np.ndarray,
    cluster_labels: np.ndarray,
    budget: int,
    min_cluster_size: int = 5
):
    """
    Modified TPCRP:
    allocate selections approximately proportional to cluster size,
    then pick the most typical points inside each cluster.
    """
    selected_indices = []

    unique_clusters = np.unique(cluster_labels)

    cluster_to_indices = {}
    cluster_sizes = {}
    for cluster_id in unique_clusters:
        indices = np.where(cluster_labels == cluster_id)[0]
        if len(indices) >= min_cluster_size:
            cluster_to_indices[cluster_id] = indices
            cluster_sizes[cluster_id] = len(indices)

    if not cluster_to_indices:
        return selected_indices

    total_points = sum(cluster_sizes.values())

    # Initial allocation proportional to cluster size
    allocation = {}
    remainders = []

    allocated_total = 0
    for cluster_id, size in cluster_sizes.items():
        exact_share = budget * (size / total_points)
        base_share = int(np.floor(exact_share))
        allocation[cluster_id] = base_share
        allocated_total += base_share
        remainders.append((cluster_id, exact_share - base_share))

    remaining_budget = budget - allocated_total
    remainders.sort(key=lambda x: x[1], reverse=True)
```

```python
    remaining_budget = budget - allocated_total
    remainders.sort(key=lambda x: x[1], reverse=True)

    for i in range(remaining_budget):
        cluster_id = remainders[i % len(remainders)][0]
        allocation[cluster_id] += 1

    for cluster_id in allocation:
        if allocation[cluster_id] == 0 and len(cluster_to_indices[cluster_id]) >= min_cluster_size:
            allocation[cluster_id] = 1

    while sum(allocation.values()) > budget:
        removable = [cid for cid, count in allocation.items() if count > 1]
        if not removable:
            break
        smallest = min(removable, key=lambda cid: cluster_sizes[cid])
        allocation[smallest] -= 1

    # Select top typical points within each cluster
    for cluster_id, num_to_select in allocation.items():
        if num_to_select <= 0:
            continue

        cluster_indices = cluster_to_indices[cluster_id]
        cluster_features = features[cluster_indices]

        k = min(20, len(cluster_indices) - 1)
        if k < 1:
            continue

        scores = compute_typicality_of_points(cluster_features, k=k)
        ranked_local_indices = np.argsort(scores)[::-1]  # descending

        take = min(num_to_select, len(ranked_local_indices))
        chosen_local = ranked_local_indices[:take]
        chosen_global = cluster_indices[chosen_local]

        selected_indices.extend(chosen_global.tolist())

    selected_indices = selected_indices[:budget]

    return selected_indices


def select_centrality_typical_samples(features, cluster_labels, budget, alpha=0.7):
    """
    Modified TPCRP:
    combine typicality + centrality score
    """

    selected_indices = []

    unique_clusters = np.unique(cluster_labels)

    for cluster_id in unique_clusters:

        cluster_indices = np.where(cluster_labels == cluster_id)[0]

        if len(cluster_indices) == 0:
            continue

        cluster_features = features[cluster_indices]

        typicality = compute_typicality_of_points(cluster_features)

        centrality = compute_centrality(cluster_features)

        # normalize scores
        typicality = (typicality - typicality.min()) / (typicality.max() - typicality.min() + 1e-8)
        centrality = (centrality - centrality.min()) / (centrality.max() - centrality.min() + 1e-8)

        score = alpha * typicality + (1 - alpha) * centrality

        best_index_local = np.argmax(score)

        best_index_global = cluster_indices[best_index_local]

        selected_indices.append(best_index_global)

    return selected_indices[:budget]
```

9. Random baseline

```python
def select_random_samples(dataset_size: int, budget: int, seed: int = 42):
    rng = np.random.default_rng(seed)
    selected_indices = rng.choice(dataset_size, size=budget, replace=False)
    return selected_indices.tolist()
```

10. Test loader

```python
def get_cifar10_test_loader(batch_size: int = 256):
    transform = transforms.Compose([
        transforms.Resize((96, 96)),
        transforms.ToTensor(),
    ])

    dataset = datasets.CIFAR10(
        root="./data",
        train=False,
        download=True,
        transform=transform
    )

    loader = DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=False
    )

    return dataset, loader
```

11. Downstream classifier training

```python
def get_device():
    if torch.backends.mps.is_available():
        return "mps"
    if torch.cuda.is_available():
        return "cuda"
    return "cpu"


def create_selected_subset(dataset, selected_indices):
    return Subset(dataset, selected_indices)


def create_model(num_classes: int = 10):
    model = models.resnet18(weights=None)
    model.fc = nn.Linear(model.fc.in_features, num_classes)
    return model


def train_model(train_dataset, test_loader, epochs: int = 5, batch_size: int = 32):
    device = get_device()
    print("Training device:", device)

    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)

    model = create_model(num_classes=10).to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.Adam(model.parameters(), lr=0.001)

    for epoch in range(epochs):
        model.train()
        running_loss = 0.0
        correct = 0
        total = 0

        for images, labels in train_loader:
            images, labels = images.to(device), labels.to(device)

            optimizer.zero_grad()
            outputs = model(images)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss = running_loss + loss.item()
            _, predicted = torch.max(outputs, 1)
            total = total + labels.size(0)
            correct = correct + (predicted == labels).sum().item()

        train_acc = 100.0 * correct / total if total > 0 else 0.0
        print(
            f"Epoch {epoch + 1}/{epochs} | "
            f"Loss: {running_loss:.4f} | "
            f"Train Acc: {train_acc:.2f}%"
        )

    test_acc = evaluate_model(model, test_loader, device)
    return model, test_acc


def evaluate_model(model, test_loader, device):
    model.eval()
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)

            outputs = model(images)
            _, predicted = torch.max(outputs, 1)

            total += labels.size(0)
            correct += (predicted == labels).sum().item()

    accuracy = 100.0 * correct / total if total > 0 else 0.0
    print(f"Test Accuracy: {accuracy:.2f}%")
    return accuracy
```

13. Original TPCRP pipeline

```python
SUBSET_SIZE = 50000
BUDGET = 200
EPOCHS = 30
SIMCLR_EPOCHS = 30
NUM_RUNS = 10 #5

MODEL_PATH = "models/simclr_resnet18.pth"


def run_tpcrp(dataset, features, test_loader):
    print("\n--- Running TPCRP ---")

    print("1. Clustering SimCLR embeddings...")
    cluster_labels = cluster_features(features, num_clusters=BUDGET)

    print("2. Selecting most typical sample per cluster...")
    selected_indices = select_most_typical_per_cluster(
        features,
        cluster_labels,
        budget=BUDGET
    )

    print(f"3. Number of selected samples: {len(selected_indices)}")

    selected_labels = [dataset[i][1] for i in selected_indices]
    print("4. Selected labels:", selected_labels)

    train_subset = create_selected_subset(dataset, selected_indices)

    print("5. Training classifier on TPCRP samples...")
    _, test_acc = train_model(train_subset, test_loader, EPOCHS, batch_size=16)

    return selected_indices, test_acc
```

14. Modified centrality based typicality TPCRP pipeline

```python
def run_centrality_tpcrp(dataset, features, test_loader):

    print("\n--- Running Centrality-Aware TPCRP ---")

    print("1. Clustering embeddings...")
    cluster_labels = cluster_features(features, num_clusters=BUDGET)

    print("2. Selecting centrality-weighted typical samples...")
    selected_indices = select_centrality_typical_samples(
        features,
        cluster_labels,
        budget=BUDGET
    )

    print(f"3. Number of selected samples: {len(selected_indices)}")

    selected_labels = [dataset[i][1] for i in selected_indices]
    print("4. Selected labels:", selected_labels)

    train_subset = create_selected_subset(dataset, selected_indices)

    print("5. Training classifier...")
    _, test_acc = train_model(train_subset, test_loader, EPOCHS, batch_size=16)

    return selected_indices, test_acc
```

14. Random baseline

```python
def run_random_baseline(dataset, test_loader):
    print("\n--- Running Random Baseline ---")

    selected_indices = select_random_samples(len(dataset), BUDGET, seed=42)

    print(f"1. Number of selected samples: {len(selected_indices)}")

    selected_labels = [dataset[i][1] for i in selected_indices]
    print("2. Selected labels:", selected_labels)

    train_subset = create_selected_subset(dataset, selected_indices)

    print("3. Training classifier on random samples...")
    _, test_acc = train_model(train_subset, test_loader, EPOCHS, batch_size=16)

    return selected_indices, test_acc
```

15. Main experiment execution

```python
_, test_loader = get_cifar10_test_loader()

results_dir = project_root / "results"
models_dir = project_root / "models"
results_dir.mkdir(exist_ok=True)
models_dir.mkdir(exist_ok=True)

MODEL_PATH = project_root / "models" / "simclr_resnet18.pth"
print("Using model path:", MODEL_PATH)
print("Model exists:", MODEL_PATH.exists())

if not MODEL_PATH.exists():
    train_simclr(
        epochs=SIMCLR_EPOCHS,
        batch_size=128,
        lr=1e-3,
        temperature=0.5,
        save_path=str(MODEL_PATH)
    )

dataset, features = extract_embeddings(str(MODEL_PATH), train=True)
features = features[:SUBSET_SIZE]
dataset = Subset(dataset, list(range(SUBSET_SIZE)))

tpcrp_indices, tpcrp_acc = run_tpcrp(dataset, features, test_loader)
centrality_indices, centrality_acc = run_centrality_tpcrp(dataset, features, test_loader)
random_indices, random_acc = run_random_baseline(dataset, test_loader)


print("Original TPCRP Accuracy:", tpcrp_acc)
print("Modified Centrality TPCRP Accuracy:", centrality_acc)
print("Random baseline accuracy:", random_acc)
```

```
/Users/omxr/Desktop/VSCodeProjects/5CCSAMLF/typiclust-active-learning/.venv/lib/python3.14/site-packages/torchvision/datasets/cifar.py:83: VisibleDeprecationWarning: dtype(): align should be passed as Python or NumPy boolean but got `align=0`. Did you mean to pass a tuple to create a subarray type? (Deprecated NumPy 2.4)
  entry = pickle.load(f, encoding="latin1")
Using model path: /Users/omxr/Desktop/VSCodeProjects/5CCSAMLF/typiclust-active-learning/models/simclr_resnet18.pth
Model exists: True

--- Running TPCRP ---
1. Clustering SimCLR embeddings...
2. Selecting most typical sample per cluster...
3. Number of selected samples: 200
4. Selected labels: [6, 8, 7, 9, 9, 0, 8, 5, 6, 1, 7, 0, 7, 1, 9, 1, 6, 0, 6, 6, 7, 7, 1, 6, 8, 0, 4, 1, 4, 4, 3, 9, 1, 1, 1, 2, 9, 7, 9, 0, 3, 4, 4, 2, 8, 4, 1, 7, 6, 1, 8,
4, 5, 2, 1, 8, 1, 5, 1, 5, 5, 7, 5, 6, 6, 1, 1, 0, 8, 4, 8, 4, 2, 1, 7, 7, 0, 0, 9, 2, 6, 1, 7, 5, 2, 1, 5, 2, 3, 6, 1, 1, 8, 6, 4, 5, 7, 7, 2, 9, 2, 1, 5, 9, 7, 4, 1, 5, 4,
9, 0, 7, 3, 6, 2, 2, 6, 8, 1, 2, 6, 4, 4, 7, 6, 6, 1, 4, 9, 5, 5, 7, 9, 6, 2, 6, 1, 9, 6, 6, 3, 7, 4, 0, 6, 4, 4, 2, 6, 4, 4, 5, 9, 8, 5, 3, 6, 6, 7, 1, 4, 5, 8, 3, 3, 5, 1,
0, 0, 4, 7, 0, 2, 2, 5, 3, 4, 8, 2, 7, 2, 3, 8, 6, 4, 0, 4, 5, 8, 4, 9, 4, 0, 0, 4, 4, 0, 2, 1, 1]
5. Training classifier on TPCRP samples...
Training device: mps
Epoch 1/30 | Loss: 35.0689 | Train Acc: 15.50%
Epoch 2/30 | Loss: 24.9307 | Train Acc: 31.50%
Epoch 3/30 | Loss: 15.0475 | Train Acc: 58.50%
Epoch 4/30 | Loss: 11.6930 | Train Acc: 70.00%
Epoch 5/30 | Loss: 9.3743 | Train Acc: 75.50%
Epoch 6/30 | Loss: 7.6542 | Train Acc: 79.00%
Epoch 7/30 | Loss: 9.6383 | Train Acc: 76.00%
Epoch 8/30 | Loss: 8.6425 | Train Acc: 80.50%
Epoch 9/30 | Loss: 6.5993 | Train Acc: 82.50%
Epoch 10/30 | Loss: 5.9317 | Train Acc: 86.00%
Epoch 11/30 | Loss: 5.8428 | Train Acc: 83.50%
Epoch 12/30 | Loss: 5.4043 | Train Acc: 86.50%
Epoch 13/30 | Loss: 4.3304 | Train Acc: 92.00%
Epoch 14/30 | Loss: 5.3192 | Train Acc: 89.00%
Epoch 15/30 | Loss: 4.4772 | Train Acc: 90.50%
Epoch 16/30 | Loss: 3.6475 | Train Acc: 92.50%
Epoch 17/30 | Loss: 3.3915 | Train Acc: 92.50%
```

```
Epoch 19/30 | Loss: 3.8139 | Train Acc: 91.50%
Epoch 20/30 | Loss: 3.2344 | Train Acc: 91.50%
Epoch 21/30 | Loss: 2.3851 | Train Acc: 94.50%
Epoch 22/30 | Loss: 2.9919 | Train Acc: 92.50%
Epoch 23/30 | Loss: 2.2592 | Train Acc: 94.50%
Epoch 24/30 | Loss: 1.3855 | Train Acc: 96.00%
Epoch 25/30 | Loss: 1.6243 | Train Acc: 97.00%
Epoch 26/30 | Loss: 2.4757 | Train Acc: 95.00%
Epoch 27/30 | Loss: 1.8139 | Train Acc: 97.00%
Epoch 28/30 | Loss: 1.8728 | Train Acc: 97.50%
Epoch 29/30 | Loss: 2.6364 | Train Acc: 94.50%
Epoch 30/30 | Loss: 2.8848 | Train Acc: 94.50%
Test Accuracy: 12.06%

--- Running Centrality-Aware TPCRP ---
1. Clustering embeddings...
2. Selecting centrality-weighted typical samples...
3. Number of selected samples: 200
4. Selected labels: [1, 9, 9, 3, 5, 9, 5, 7, 0, 2, 7, 1, 8, 5, 4, 6, 8, 2, 4, 6, 2, 7, 0, 5, 8, 5, 3, 9, 7, 7, 7, 0, 9, 5, 8, 7, 8, 5, 6, 7, 0, 0, 7, 6, 5, 1, 9, 9, 1, 7, 3,
9, 8, 2, 5, 5, 3, 4, 1, 4, 4, 2, 6, 9, 0, 8, 3, 0, 6, 0, 8, 6, 4, 4, 8, 2, 2, 4, 4, 3, 0, 8, 9, 4, 7, 4, 8, 4, 0, 7, 2, 6, 7, 2, 4, 9, 9, 4, 5, 2, 8, 8, 4, 9, 4, 1, 8, 0, 8,
6, 9, 9, 1, 8, 5, 0, 0, 5, 8, 4, 4, 6, 7, 5, 0, 8, 9, 4, 5, 2, 7, 9, 9, 6, 1, 7, 8, 7, 3, 3, 6, 2, 4, 0, 0, 1, 4, 2, 8, 6, 1, 4, 1, 5, 6, 7, 1, 1, 1, 4, 0, 3, 4, 3, 7, 7, 7,
6, 3, 0, 6, 9, 1, 5, 3, 9, 9, 4, 6, 5, 4, 0, 3, 0, 3, 0, 5, 1, 0, 1, 1, 5, 5, 5, 9, 0, 0, 0, 2, 0]
5. Training classifier...
Training device: mps
Epoch 1/30 | Loss: 31.9265 | Train Acc: 20.50%
Epoch 2/30 | Loss: 23.3836 | Train Acc: 45.00%
Epoch 3/30 | Loss: 14.5796 | Train Acc: 63.50%
Epoch 4/30 | Loss: 8.3656 | Train Acc: 76.00%
Epoch 5/30 | Loss: 9.7163 | Train Acc: 77.50%
Epoch 6/30 | Loss: 6.3759 | Train Acc: 86.50%
Epoch 7/30 | Loss: 5.7555 | Train Acc: 84.50%
Epoch 8/30 | Loss: 7.6081 | Train Acc: 84.50%
Epoch 9/30 | Loss: 7.7340 | Train Acc: 80.50%
Epoch 10/30 | Loss: 5.6605 | Train Acc: 86.50%
Epoch 11/30 | Loss: 3.9635 | Train Acc: 93.50%
Epoch 12/30 | Loss: 4.3382 | Train Acc: 88.50%
Epoch 13/30 | Loss: 3.6094 | Train Acc: 91.50%
Epoch 14/30 | Loss: 2.7204 | Train Acc: 92.00%
Epoch 15/30 | Loss: 3.3070 | Train Acc: 90.00%
Epoch 16/30 | Loss: 3.8244 | Train Acc: 87.50%
Epoch 17/30 | Loss: 4.2144 | Train Acc: 88.50%
Epoch 18/30 | Loss: 3.4274 | Train Acc: 91.00%
Epoch 19/30 | Loss: 4.4780 | Train Acc: 91.50%
Epoch 20/30 | Loss: 4.7099 | Train Acc: 90.00%
Epoch 21/30 | Loss: 4.3497 | Train Acc: 89.00%
Epoch 22/30 | Loss: 3.5735 | Train Acc: 93.00%
Epoch 23/30 | Loss: 2.3060 | Train Acc: 95.50%
Epoch 24/30 | Loss: 2.3377 | Train Acc: 94.50%
Epoch 25/30 | Loss: 1.6905 | Train Acc: 97.00%
Epoch 26/30 | Loss: 0.7513 | Train Acc: 98.00%
Epoch 27/30 | Loss: 1.4604 | Train Acc: 96.00%
Epoch 28/30 | Loss: 0.6888 | Train Acc: 98.00%
Epoch 29/30 | Loss: 1.9779 | Train Acc: 96.00%
Epoch 30/30 | Loss: 2.7200 | Train Acc: 94.50%
Test Accuracy: 10.57%

--- Running Random Baseline ---
1. Number of selected samples: 200
2. Selected labels: [3, 5, 5, 7, 7, 5, 1, 8, 2, 1, 9, 8, 3, 3, 8, 8, 3, 9, 3, 7, 8, 8, 3, 9, 8, 8, 9, 8, 3, 1, 8, 2, 8, 3, 1, 4, 4, 8, 9, 1, 5, 7, 8, 9, 0, 3, 3, 8, 1, 7, 7,
9, 7, 1, 3, 8, 5, 0, 0, 2, 8, 8, 6, 6, 3, 8, 5, 4, 5, 2, 7, 1, 9, 9, 7, 1, 1, 1, 7, 8, 5, 5, 5, 3, 3, 0, 0, 0, 4, 4, 9, 8, 5, 1, 9, 2, 5, 1, 6, 2, 2, 3, 9, 5, 3, 0, 3, 3, 9,
5, 6, 9, 6, 8, 1, 6, 7, 8, 2, 7, 3, 2, 9, 9, 2, 8, 0, 4, 6, 3, 5, 5, 3, 6, 5, 2, 2, 5, 8, 4, 6, 4, 5, 3, 6, 5, 2, 3, 8, 2, 4, 2, 3, 5, 5, 6, 9, 6, 9, 8, 2, 2, 3, 8, 6, 3, 1,
7, 7, 7, 2, 9, 4, 3, 8, 7, 8, 4, 0, 1, 1, 6, 0, 9, 9, 1, 3, 2, 4, 4, 2, 0, 1, 8, 5, 3, 5, 2, 4, 0]
3. Training classifier on random samples...
Training device: mps
```